

LightGBM 数学推导

从残差、 g, h 到节点目标、 w 、Gain 与区域 R

摘要： 本讲义沿一条连续主线推导 LightGBM。已有 $m - 1$ 棵树时，我们增加第 m 棵树。新树同时包含叶节点区域 $R_1, \dots, R_J$ 与叶节点输出 $w_1, \dots, w_J$ 。损失函数在当前预测处展开后，一阶导数 g 描述修正方向，二阶导数 h 描述局部曲率。把同一叶节点内的样本聚合起来，就得到节点目标函数；对它求导可得最优 w_j ，比较分裂前后的最优目标值则得到 Gain。平方损失下，每个叶节点拟合该区域内的平均残差。最后，讲义分别说明固定树结构、单步下降与完整 Boosting 过程能够获得什么收敛保证，以及这些保证为何不等于全局最优或实盘有效。

全文路线

$$\hat{y}_n^{(m-1)} \rightarrow f_m(\mathbf{x}) = \sum_j w_j \mathbb{1}(\mathbf{x} \in R_j) \rightarrow (g_n, h_n) \rightarrow \mathcal{L}_j(w_j) \rightarrow w_j^* \rightarrow \text{Gain} \rightarrow R_j \quad (1)$$

目录

目录	1	10.7 为什么没有全局最优树保证	12
1 符号与问题设置	2	10.8 必须区分三种“收敛”	13
2 第一步：增加第 m 棵树	2	11 与 Qlib 股票预测的对应关系	13
2.1 已有模型	2	12 容易混淆的三个问题	14
2.2 一棵树的两个未知部分	2	12.1 LightGBM 是否同时求 w 和 R	14
3 第二步：从损失函数得到 g 和 h	3	12.2 g, h 是模型参数吗	14
3.1 第 m 轮的原始目标	3	12.3 Gain 是熵或基尼指数吗	14
3.2 一阶导数 g_n	3	13 一页公式总结	15
3.3 二阶导数 h_n	4		
3.4 二阶泰勒展开	4		
4 平方损失下， g 和 h 就是残差语言	4		
4.1 直接求导	4		
4.2 不用泰勒公式也能看见它们	5		
5 第三步：节点目标函数从哪里来	5		
5.1 把属于同一叶节点的样本放在一起	5		
5.2 求最优叶节点输出 w_j	6		
6 从节点最优值到 Gain	6		
6.1 一个节点的最优得分	6		
6.2 一个候选分裂的 Gain	7		
7 一个完整的平方损失数值例子	7		
7.1 不分裂	8		
7.2 候选分裂	8		
8 完整算法：把各部分重新连起来	8		
9 学习率、正则化与停止条件	9		
9.1 学习率	9		
9.2 L2 正则化	9		
9.3 结构约束	9		
10 收敛性：哪些有保证，哪些没有	10		
10.1 固定区域 R_j 时， w_j 有唯一最优解	10		
10.2 正 Gain 提供单步下降保证	10		
10.3 多棵树是函数空间中的近似梯度下降	11		
10.4 平方损失下的精确下降条件	11		
10.5 当新树是残差的投影	12		
10.6 训练目标何时整体收敛	12		

1 符号与问题设置

设训练集为

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N, \quad (2)$$

其中 $\mathbf{x}_n \in \mathbb{R}^p$ 是第 n 个样本的特征向量, y_n 是监督目标。在 Qlib 的股票预测任务中, 一个样本通常对应“某只股票在某个交易日”, $\mathbf{x}_n$ 是 Alpha158 等因子, y_n 是未来收益标签。

符号	含义
$\mathbf{x}_n$	第 n 个样本的 p 维特征
y_n	第 n 个样本的真实标签
$\hat{y}_n^{(m-1)}$	前 $m-1$ 棵树给出的当前预测
f_m	准备加入的第 m 棵回归树
R_j	第 j 个叶节点覆盖的特征空间区域
w_j	第 j 个叶节点给出的统一修正值
g_n, h_n	损失对当前预测的一阶、二阶导数
G_j, H_j	叶节点内 g_n, h_n 的聚合量

表 1 核心符号表

先区分两种权重

本文的 w_j 是树叶节点的输出, 不是投资组合权重。LightGBM 先生成股票预测分数, Qlib 的交易策略再把分数转换成持仓权重。两者属于不同阶段。

2 第一步：增加第 m 棵树

2.1 已有模型

已有 $m-1$ 棵树时, 模型为

$$F_{m-1}(\mathbf{x}) = \sum_{q=1}^{m-1} f_q(\mathbf{x}). \quad (3)$$

第 n 个样本的当前预测是

$$\hat{y}_n^{(m-1)} = F_{m-1}(\mathbf{x}_n). \quad (4)$$

现在加入第 m 棵树:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta f_m(\mathbf{x}), \quad (5)$$

其中 $\eta \in (0, 1]$ 是学习率。为了看清主干推导, 下面先令 $\eta = 1$; 最后再把学习率乘回去。

2.2 一棵树的两个未知部分

具有 J 个叶节点的回归树可以写成

$$f_m(\mathbf{x}) = \sum_{j=1}^J w_j \mathbb{1}(\mathbf{x} \in R_j). \quad (6)$$

这里确实有两类未知量：

- 树结构： $R_1, \dots, R_J$ ；
- 叶节点输出： $w_1, \dots, w_J$ 。

如果 $\mathbf{x}_n \in R_j$ ，则

$$f_m(\mathbf{x}_n) = w_j, \quad (7)$$

所以新预测为

$$\hat{y}_n^{(m)} = \hat{y}_n^{(m-1)} + w_j. \quad (8)$$

直觉 R_j 回答“哪些样本放在一起”， w_j 回答“这组样本统一修正多少”。LightGBM 并不全局同时枚举所有树，而是贪心构造 R_j ；一旦某个候选区域确定， w_j 可以解析求出。

3 第二步：从损失函数得到 g 和 h

3.1 第 m 轮的原始目标

加入新树以后，希望最小化

$$\mathcal{L}^{(m)} = \sum_{n=1}^N \ell(y_n, \hat{y}_n^{(m-1)} + f_m(\mathbf{x}_n)) + \Omega(f_m). \quad (9)$$

为了把结构代价和叶节点正则写进同一个目标，本讲义采用常见的 XGBoost 风格约定：

$$\Omega(f_m) = \gamma J + \frac{\lambda}{2} \sum_{j=1}^J w_j^2. \quad (10)$$

公式尺度约定

本讲义把节点目标的实际下降量定义为 $\frac{1}{2} \frac{G^2}{H+\lambda}$ ，所以 Gain 公式含有系数 $\frac{1}{2}$ 。LightGBM 源码在未启用 L1、叶输出限制、路径平滑和单调约束时，叶值仍为 $-\frac{G}{H+\lambda_2}$ ，但它把叶得分记为 $\frac{G^2}{H+\lambda_2}$ 。因此，当分裂门槛均为零时，LightGBM 模型中的 `split_gain` 是本讲义 Gain 的两倍；两种尺度不会改变候选分裂的排序。本文的 γ 表示新增叶节点的概念性代价；若要与 LightGBM 的 `min_gain_to_split` 使用相同门槛，应按上述二倍尺度换算。

直接优化这个式子困难，因为树结构是离散对象。LightGBM 的关键动作是：在当前预测 $\hat{y}_n^{(m-1)}$ 附近，把每个样本的损失写成关于“新增修正量” $f_m(\mathbf{x}_n)$ 的二次函数。

3.2 一阶导数 g_n

对第 n 个样本，把损失写成当前预测 z 的函数：

$$L_n(z) = \ell(y_n, z). \quad (11)$$

当前所在位置是

$$z = \hat{y}_n^{(m-1)}. \quad (12)$$

定义一阶导数

$$g_n = \frac{\partial \ell(y_n, \hat{y}_n)}{\partial \hat{y}_n} \Big|_{\hat{y}_n = \hat{y}_n^{(m-1)}}. \quad (13)$$

g_n 是损失曲线在当前位置的斜率。若 $g_n > 0$ ，增加预测会增加损失，应向下修正；若 $g_n < 0$ ，增加预测会降低损失，应向上修正。因此真正的下降方向是 $-g_n$ 。

3.3 二阶导数 h_n

定义二阶导数

$$h_n = \frac{\partial^2 \ell(y_n, \hat{y}_n)}{\partial \hat{y}_n^2} \Big|_{\hat{y}_n = \hat{y}_n^{(m-1)}}. \quad (14)$$

h_n 描述损失曲线的局部曲率：它衡量梯度随预测值变化得有多快。在一维牛顿法中，局部最优修正是

$$\Delta^* = -\frac{g}{h}. \quad (15)$$

这已经预告了 LightGBM 叶节点输出公式为什么会出现“梯度和除以 Hessian 和”。

3.4 二阶泰勒展开

设新增修正量为

$$\Delta_n = f_m(\mathbf{x}_n). \quad (16)$$

在当前预测处做二阶泰勒展开：

$$\ell(y_n, \hat{y}_n^{(m-1)} + \Delta_n) \approx \ell(y_n, \hat{y}_n^{(m-1)}) + g_n \Delta_n + \frac{1}{2} h_n \Delta_n^2. \quad (17)$$

去掉与新树无关的常数项后，第 m 棵树要优化

$$\tilde{\mathcal{L}}^{(m)} = \sum_{n=1}^N \left[g_n f_m(\mathbf{x}_n) + \frac{1}{2} h_n f_m(\mathbf{x}_n)^2 \right] + \Omega(f_m). \quad (18)$$

g, h 的最短解释

g_n 给出第 n 个样本应该向哪个方向修正； h_n 给出损失在当前位置有多弯，从而缩放修正步长。二者只是损失函数局部二次近似的两个系数。

4 平方损失下， g 和 h 就是残差语言

4.1 直接求导

采用带 $\frac{1}{2}$ 的平方损失：

$$\ell(y_n, \hat{y}_n) = \frac{1}{2} (y_n - \hat{y}_n)^2. \quad (19)$$

一阶导数是

$$g_n = \hat{y}_n^{(m-1)} - y_n. \quad (20)$$

定义当前残差

$$r_n = y_n - \hat{y}_n^{(m-1)}, \quad (21)$$

则

$$(g_n = -r_n). \quad (22)$$

二阶导数是

$$(h_n = 1). \quad (23)$$

因此平方损失下，负梯度就是残差：

$$-g_n = r_n = y_n - \hat{y}_n^{(m-1)}. \quad (24)$$

4.2 不用泰勒公式也能看见它们

加入新树修正 $\Delta_n = f_m(\mathbf{x}_n)$ 后，新的残差为

$$r_n - \Delta_n. \quad (25)$$

平方损失可以精确展开为

$$\frac{1}{2}(r_n - \Delta_n)^2 = \frac{1}{2}r_n^2 - r_n\Delta_n + \frac{1}{2}\Delta_n^2. \quad (26)$$

又因为

$$g_n = -r_n, \quad h_n = 1, \quad (27)$$

所以

$$\frac{1}{2}(r_n - \Delta_n)^2 = \underbrace{\frac{1}{2}r_n^2}_{\text{旧模型常数}} + g_n\Delta_n + \frac{1}{2}h_n\Delta_n^2. \quad (28)$$

直觉 对平方损失，二阶展开不是近似，而是精确等式。此时可以完全忘掉抽象的 Hessian： g_n 就是负残差， h_n 恒为 1。

5 第三步：节点目标函数从哪里来

5.1 把属于同一叶节点的样本放在一起

设第 j 个叶节点包含的样本索引集合为

$$I_j = \{n : \mathbf{x}_n \in R_j\}. \quad (29)$$

对于所有 $n \in I_j$ ，树的输出都相同：

$$f_m(\mathbf{x}_n) = w_j. \quad (30)$$

因此这些样本对二阶近似目标的贡献为

$$\sum_{n \in I_j} \left[g_n w_j + \frac{1}{2} h_n w_j^2 \right]. \quad (31)$$

定义节点聚合量

$$\left(G_j = \sum_{n \in I_j} g_n \right), \quad \left(H_j = \sum_{n \in I_j} h_n \right). \quad (32)$$

因为同一节点内的 w_j 是共同常数，所以可以提出求和号：

$$\sum_{n \in I_j} g_n w_j = G_j w_j, \quad (33)$$

$$\sum_{n \in I_j} \frac{1}{2} h_n w_j^2 = \frac{1}{2} H_j w_j^2. \quad (34)$$

再加入 L2 正则项 $\lambda \frac{w_j^2}{2}$, 得到第 j 个节点的目标函数:

$$\left(\tilde{\mathcal{L}}_j(w_j) = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right). \quad (35)$$

整棵树的目标是

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^J \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma J. \quad (36)$$

5.2 求最优叶节点输出 w_j

给定区域 R_j 后, G_j, H_j 已知。对节点目标求导:

$$\frac{\partial \tilde{\mathcal{L}}_j}{\partial w_j} = G_j + (H_j + \lambda) w_j. \quad (37)$$

令导数等于零:

$$G_j + (H_j + \lambda) w_j = 0. \quad (38)$$

得到

$$\left(w_j^* = -\frac{G_j}{H_j + \lambda} \right). \quad (39)$$

平方损失下, $g_n = -r_n$ 、 $h_n = 1$, 所以

$$G_j = -\sum_{n \in I_j} r_n, \quad H_j = |I_j|. \quad (40)$$

于是

$$w_j^* = \frac{\sum_{n \in I_j} r_n}{|I_j| + \lambda}. \quad (41)$$

当 $\lambda = 0$ 时:

$$\left(w_j^* = \frac{1}{|I_j|} \sum_{n \in I_j} (y_n - \hat{y}_n^{(m-1)}) \right). \quad (42)$$

也就是说, 叶节点输出就是该节点内的平均残差。

节点目标函数的来源

节点目标函数不是额外假设。它来自三步: 先在当前预测处对每个样本的损失做二阶展开; 再利用同一叶节点的样本共享 w_j ; 最后把节点内的 g_n, h_n 分别求和为 G_j, H_j 。

6 从节点最优值到 Gain

6.1 一个节点的最优得分

将

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (43)$$

代回节点目标:

$$\tilde{\mathcal{L}}_j(w_j^*) = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}. \quad (44)$$

定义节点能够带来的目标下降量为

$$\text{Score}(I_j) = \frac{1}{2} \frac{G_j^2}{H_j + \lambda}. \quad (45)$$

注意符号关系: 最优目标贡献是 $-\text{Score}$, 所以 Score 越大, 节点通过独立输出 w_j 能降低的损失越多。

6.2 一个候选分裂的 Gain

设父节点 I_P 被候选条件 $x^{(k)} \leq s$ 分成

$$I_L = \{n \in I_P : x_n^{(k)} \leq s\}, \quad (46)$$

$$I_R = \{n \in I_P : x_n^{(k)} > s\}. \quad (47)$$

统计量满足

$$G_P = G_L + G_R, \quad H_P = H_L + H_R. \quad (48)$$

在本讲义的目标下降量尺度下, 分裂收益等于“两个子节点的最优得分”减去“父节点的最优得分”, 再减去新增叶节点的复杂度代价:

$$\left(\text{Gain}(k, s) = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma \right). \quad (49)$$

LightGBM 在候选特征和阈值中选择

$$(k^*, s^*) = \arg \max_{k, s} \text{Gain}(k, s). \quad (50)$$

由这个最佳切分得到两个新区域:

$$R_L = R_P \cap \{\mathbf{x} : x^{(k^*)} \leq s^*\}, \quad (51)$$

$$R_R = R_P \cap \{\mathbf{x} : x^{(k^*)} > s^*\}. \quad (52)$$

递归分裂后, 每个叶节点区域都是路径上若干切分条件的交集:

$$R_j = \bigcap_{q \in \mathcal{P}_j} \{\mathbf{x} : x^{(k_q)} \leq s_q \text{ 或 } x^{(k_q)} > s_q\}. \quad (53)$$

因此, R_j 不是由一个连续方程解析求解, 而是由最大 Gain 的贪心切分逐步生成。

7 一个完整的平方损失数值例子

设一个父节点中有四个样本。当前模型的预测与真实值形成残差

$$\mathbf{r} = (3, 2, -2, -3). \quad (54)$$

平方损失下

$$\mathbf{g} = -\mathbf{r} = (-3, -2, 2, 3), \quad \mathbf{h} = (1, 1, 1, 1). \quad (55)$$

暂令

$$\lambda = 0, \quad \gamma = 0. \quad (56)$$

7.1 不分裂

父节点统计量为

$$G_P = -3 - 2 + 2 + 3 = 0, \quad H_P = 4. \quad (57)$$

父节点最优输出:

$$w_P^* = -\frac{0}{4} = 0. \quad (58)$$

父节点得分:

$$\text{Score}(I_P) = \frac{1}{2} \frac{0^2}{4} = 0. \quad (59)$$

这说明正负残差在同一个节点中完全抵消，一个统一的 w_P 无法同时修正它们。

7.2 候选分裂

假设某个特征阈值把样本分成

$$I_L : \{g_1, g_2\} = (-3, -2), \quad (60)$$

$$I_R : \{g_3, g_4\} = (2, 3). \quad (61)$$

则

$$G_L = -5, \quad H_L = 2, \quad G_R = 5, \quad H_R = 2. \quad (62)$$

左右叶节点输出为

$$w_L^* = -\frac{-5}{2} = 2.5, \quad w_R^* = -\frac{5}{2} = -2.5. \quad (63)$$

在本讲义的尺度下，Gain 为

$$\text{Gain} = \frac{1}{2} \left[\frac{25}{2} + \frac{25}{2} - \frac{0}{4} \right] = 12.5. \quad (64)$$

若使用 LightGBM 源码的 `split_gain` 尺度，并令 `min_gain_to_split = 0`，同一候选分裂报告的数值是 25。两者只相差常数倍，不影响它在候选切分中的排名。

直觉 这个切分把“需要向上修正”的样本与“需要向下修正”的样本分开了。父节点内相互抵消的梯度，进入两个子节点后不再抵消，所以 Gain 很大。

8 完整算法：把各部分重新连起来

第 m 轮训练一棵树时，可以按下面的顺序理解：

输入：样本 $(\mathbf{x}_n, y_n)$ ，已有模型 $F_{(m-1)}$

1. 计算当前预测

$$\hat{y}_n = F_{(m-1)}(\mathbf{x}_n)$$

2. 计算每个样本的局部损失信息

$$\begin{aligned} g_n &= d \text{ loss} / d y_{\text{hat}_n} \\ h_n &= d^2 \text{ loss} / d y_{\text{hat}_n}^2 \end{aligned}$$

3. 从根节点开始，枚举候选特征和阈值
对每个候选分裂聚合 G_L, H_L, G_R, H_R
4. 计算每个候选分裂的 Gain
选择 Gain 最大的分裂，生成新的区域 R_L, R_R
5. 按 leaf-wise 规则继续分裂当前收益最大的叶节点
直到叶节点数、深度、最小样本数或最小 Gain 触发停止
6. 对每个最终区域 R_j 计算
 $w_j = -G_j / (H_j + \text{lambda})$
7. 得到新树
 $f_m(x) = \sum_j w_j 1(x \text{ in } R_j)$
8. 更新模型
 $F_m(x) = F_{(m-1)}(x) + \text{eta } f_m(x)$

需要注意，算法在搜索切分时已经把候选子节点的最优 w_L, w_R 代入目标，因此它能够比较不同候选区域“各自使用最优叶值以后”能降低多少损失。

9 学习率、正则化与停止条件

9.1 学习率

树计算出的叶节点值是 w_j ，但模型实际加入的是

$$\eta w_j. \quad (65)$$

因此

$$\hat{y}_n^{(m)} = \hat{y}_n^{(m-1)} + \eta w_j, \quad \mathbf{x}_n \in R_j. \quad (66)$$

较小的 η 让每棵树只完成一部分修正，通常需要更多树，但能降低过拟合风险。

9.2 L2 正则化

λ 出现在分母中：

$$w_j^* = -\frac{G_j}{H_j + \lambda}. \quad (67)$$

当 λ 增大时，叶节点输出绝对值被压缩。它也会降低小样本叶节点的 Gain，使树更保守。

9.3 结构约束

LightGBM 还通过以下约束控制区域 R_j 的复杂度：

- **num_leaves**：限制叶节点总数 J ；
- **max_depth**：限制从根到叶的路径长度；
- **min_data_in_leaf**：要求 $|I_j|$ 不得过小；
- **min_gain_to_split**：要求候选 Gain 足够大；
- **feature_fraction**、**bagging_fraction**：随机使用部分特征或样本。

10 收敛性：哪些有保证，哪些没有

LightGBM 的“收敛”不能只用一句话回答。固定树结构时，叶节点值是凸问题的唯一最优解；构造树时，正 Gain 保证局部目标下降；连续增加树时，只有在损失光滑、新树确实提供下降方向且学习率合适等条件下，经验损失才有整体收敛保证。离散树结构仍由贪心搜索产生，所以这些结论不保证全局最优树，也不保证测试集或实盘收益持续改善。

分层结论

- 固定 R_j ：若 $H_j + \lambda > 0$ ， w_j^* 是唯一全局最优解；
- 固定当前轮次：正 Gain 降低二阶近似目标，平方损失下也降低真实目标；
- 多轮 Boosting：在光滑性、下降方向和步长条件下，训练目标单调下降并收敛；
- 完整 LightGBM：贪心树搜索不保证得到所有树结构中的全局最优解；
- 泛化表现：训练损失收敛不等于验证损失、测试损失或实盘表现收敛。

10.1 固定区域 R_j 时， w_j 有唯一最优解

给定叶节点区域 R_j ，节点目标是

$$\tilde{\mathcal{L}}_j(w_j) = G_j w_j + \frac{1}{2}(H_j + \lambda)w_j^2. \quad (68)$$

它对 w_j 的二阶导数为

$$\frac{\partial^2 \tilde{\mathcal{L}}_j}{\partial w_j^2} = H_j + \lambda. \quad (69)$$

只要

$$H_j + \lambda > 0, \quad (70)$$

节点目标就是严格凸函数，因此驻点

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad (71)$$

是唯一的全局最优解。这个结论只解决“区域已经给定以后，叶节点应该输出多少”，并没有解决如何全局选择 R_j 。

10.2 正 Gain 提供单步下降保证

父节点的最优目标贡献为

$$-\frac{1}{2} \frac{G_P^2}{H_P + \lambda}. \quad (72)$$

候选分裂产生左右子节点后，最优目标贡献变为

$$-\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} \right]. \quad (73)$$

若候选分裂满足

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma > 0, \quad (74)$$

那么分裂后的二阶近似正则化目标严格小于分裂前的目标。对于平方损失，二阶展开是精确等式，因此这个结论对本讲义定义的局部正则化目标也是精确的；若只讨论不带正则的经验平方损失，仍应结合实际叶值与学习率检查更新后的损失变化。对于一般损失，Gain 衡量的是当前预测附近的局部二阶模型；学习率和正则化负责限制修正幅度，使局部近似保持可靠。

10.3 多棵树是函数空间中的近似梯度下降

定义经验风险

$$\mathcal{L}(F) = \sum_{n=1}^N \ell(y_n, F(\mathbf{x}_n)). \quad (75)$$

Boosting 更新为

$$F_m = F_{m-1} + \eta f_m. \quad (76)$$

假设 $\mathcal{L}$ 的梯度是 β -Lipschitz 的，即

$$\|\nabla \mathcal{L}(F) - \nabla \mathcal{L}(G)\| \leq \beta \|F - G\|. \quad (77)$$

下降引理给出

$$\mathcal{L}(F_{m-1} + \eta f_m) \leq \mathcal{L}(F_{m-1}) + \eta \langle \nabla \mathcal{L}(F_{m-1}), f_m \rangle + \frac{\beta \eta^2}{2} \|f_m\|^2. \quad (78)$$

如果新树与负梯度方向具有正相关性：

$$\langle \nabla \mathcal{L}(F_{m-1}), f_m \rangle < 0, \quad (79)$$

并且 η 足够小，那么右侧新增的两项之和为负，因此

$$\mathcal{L}(F_m) < \mathcal{L}(F_{m-1}). \quad (80)$$

这里的新树不需要等于完整负梯度。它只需要与负梯度保持足够的对齐，就能成为一个下降方向；决策树的作用正是近似这个方向。

10.4 平方损失下的精确下降条件

定义第 $m-1$ 轮的残差向量

$$\mathbf{r}_{m-1} = \mathbf{y} - F_{m-1}(\mathbf{X}). \quad (81)$$

平方损失为

$$\mathcal{L}(F_{m-1}) = \frac{1}{2} \|\mathbf{r}_{m-1}\|^2. \quad (82)$$

加入新树以后

$$\mathbf{r}_m = \mathbf{r}_{m-1} - \eta \mathbf{f}_m, \quad (83)$$

其中 $\mathbf{f}_m$ 表示新树在全部训练样本上的输出向量。于是

$$\mathcal{L}(F_m) - \mathcal{L}(F_{m-1}) = -\eta \langle \mathbf{r}_{m-1}, \mathbf{f}_m \rangle + \frac{\eta^2}{2} \|\mathbf{f}_m\|^2. \quad (84)$$

只要新树与残差正相关：

$$\langle \mathbf{r}_{m-1}, \mathbf{f}_m \rangle > 0, \quad (85)$$

并且学习率满足

$$0 < \eta < \frac{2\langle \mathbf{r}_{m-1}, \mathbf{f}_m \rangle}{\|\mathbf{f}_m\|^2}, \quad (86)$$

就有

$$\mathcal{L}(F_m) < \mathcal{L}(F_{m-1}). \quad (87)$$

这个不等式直接说明学习率的作用：即使树找到了正确方向，步长过大仍可能越过局部最低点，使损失反而上升。

10.5 当新树是残差的投影

为了看见一个更简洁的充分条件，假设新树输出向量是残差在某个树函数空间上的正交投影：

$$\mathbf{f}_m = P_{\mathcal{T}} \mathbf{r}_{m-1}. \quad (88)$$

正交投影满足

$$\langle \mathbf{r}_{m-1}, \mathbf{f}_m \rangle = \|\mathbf{f}_m\|^2. \quad (89)$$

代入损失变化式：

$$\mathcal{L}(F_m) - \mathcal{L}(F_{m-1}) = -\eta \left(1 - \frac{\eta}{2}\right) \|\mathbf{f}_m\|^2. \quad (90)$$

因此只要

$$0 < \eta < 2, \quad (91)$$

训练损失就不会增加。实际算法使用的回归树通常只是近似投影，但这个结果解释了为什么较小的学习率能让 Boosting 迭代更稳定。

10.6 训练目标何时整体收敛

若每轮都能找到下降方向、学习率满足下降条件、损失函数存在下界，那么

$$\mathcal{L}(F_0) \geq \mathcal{L}(F_1) \geq \dots \geq \inf_F \mathcal{L}(F). \quad (92)$$

训练目标构成一个单调递减且有下界的数列，因此目标值收敛到某个有限值。这个结论本身不说明极限等于全局最优值，也不说明梯度或模型参数一定收敛。

要进一步得到收敛率，还需要一个统一的“弱学习”条件。存在常数 $\delta > 0$ ，使每一轮的树都满足

$$-\frac{\langle \nabla \mathcal{L}(F_{m-1}), \mathbf{f}_m \rangle}{\|\nabla \mathcal{L}(F_{m-1})\| \|\mathbf{f}_m\|} \geq \delta. \quad (93)$$

这个条件要求新树与负梯度的夹角余弦始终存在正下界。若目标凸且光滑，同时弱学习条件与步长规则成立，常见分析可以给出次线性收敛率；若目标强凸或满足 Polyak-Lojasiewicz 条件，在同样的方向质量和步长条件下可以得到线性收敛率。仅凭“凸且光滑”不能推出这些速率。若目标非凸，通常只能讨论梯度趋近于零或算法趋向驻点。

如果当前树空间无法继续解释残差，弱学习条件就可能失效，新树输出会接近零，训练过程也会停在该函数空间能够达到的水平。

10.7 为什么没有全局最优树保证

区域 $R_1, \dots, R_J$ 来自离散树结构。LightGBM 在每个节点选择当前 Gain 最大的特征与阈值：

$$(k^*, s^*) = \arg \max_{k, s} \text{Gain}(k, s). \quad (94)$$

这个选择只保证当前节点的局部最优分裂。一旦早期切分确定，后续区域只能在该结构上继续生长；另一条早期 Gain 略低的路径，可能最终形成整体更优的树。因此 LightGBM 不保证

$$F_M = \arg \min_{F \text{ 属于全部树集成}} \mathcal{L}(F). \quad (95)$$

它保证的是若干可计算的局部下降步骤，而不是组合树结构上的全局最优性。

10.8 必须区分三种“收敛”

对象	能够说明什么	不能推出什么
训练目标收敛	经验损失趋向有限值	不推出全局最优或泛化有效
模型参数收敛	树结构与输出最终稳定	损失收敛不必伴随参数收敛
测试或实盘收敛	未见样本上的表现稳定	不能由训练损失单独保证

表 2 三种收敛概念不能混用

分类数据完全可分时，逻辑损失可能趋近于零，而模型分数的绝对值继续增大。这说明“损失收敛”并不意味着“参数收敛”。同样，训练损失持续下降时，验证损失可能先下降后上升。这就是 Qlib 工作流使用验证集和 early stopping 的原因：

$$m^* = \arg \min_m \mathcal{L}_{\text{valid}}(F_m). \quad (96)$$

最终结论

LightGBM 对固定叶节点值有严格凸优化保证，对正 Gain 分裂有局部下降保证，对整个 Boosting 序列有依赖光滑性、弱学习条件与步长选择的训练目标收敛保证。它没有无条件的全局最优树保证，也没有从训练收敛直接推导测试集或实盘表现的保证。

11 与 Qlib 股票预测的对应关系

在当前 Qlib LightGBM 工作流中，可以把数学对象对应为：

数学对象	Qlib 中的含义
$\mathbf{x}_{i,t}$	股票 i 在交易日 t 的 Alpha158 因子
$y_{i,t}$	配置文件定义的未来收益标签
$\hat{y}_{i,t}$	LightGBM 输出的股票预测分数
R_j	满足一组因子阈值条件的股票-日期样本集合
w_j	该样本集合在当前 Boosting 轮次中的统一分数修正
f_m	第 m 棵树对全部股票-日期样本的修正函数

表 3 数学对象与 Qlib 工作流的对应

LightGBM 最终输出

$$\hat{y}_{i,t} = F_M(\mathbf{x}_{i,t}). \quad (97)$$

Qlib 再按同一天的预测分数排序，并由 TopK 等策略生成投资组合。树的 w_j 并不是最终持仓权重。

12 容易混淆的三个问题

12.1 LightGBM 是否同时求 w 和 R

从优化目标看，它们都是未知量：

$$\min_{\{R_j\}, \{w_j\}} \mathcal{L}. \quad (98)$$

但实际算法不做全局联合求解。它对每个候选分裂解析计算最优叶值，再用 Gain 贪心选择切分。可以概括为：

$$\text{候选 } R \rightarrow \text{解析最优 } w \rightarrow \text{计算 Gain} \rightarrow \text{选择最佳 } R. \quad (99)$$

12.2 g, h 是模型参数吗

不是。 g_n, h_n 是当前轮次中，每个样本的损失函数局部信息。模型更新后，它们会重新计算。平方损失下：

$$g_n = \hat{y}_n - y_n, \quad h_n = 1. \quad (100)$$

12.3 Gain 是熵或基尼指数吗

不是。LightGBM 回归树的 Gain 是二阶近似目标函数的下降量。它由 G, H 决定，而不是分类树中的信息熵。

13 一页公式总结

模型

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta f_m(\mathbf{x}), \quad (101)$$

$$f_m(\mathbf{x}) = \sum_{j=1}^J w_j \mathbb{1}(\mathbf{x} \in R_j). \quad (102)$$

样本梯度与 Hessian

$$g_n = \frac{\partial \ell}{\partial \hat{y}_n}, \quad h_n = \frac{\partial^2 \ell}{\partial \hat{y}_n^2}. \quad (103)$$

平方损失下: $g_n = \hat{y}_n - y_n = -r_n$, $h_n = 1$ 。

节点聚合与目标

$$G_j = \sum_{n \in I_j} g_n, \quad H_j = \sum_{n \in I_j} h_n, \quad (104)$$

$$\tilde{\mathcal{L}}_j(w_j) = G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2. \quad (105)$$

最优叶节点输出

$$\left(w_j^* = -\frac{G_j}{H_j + \lambda} \right). \quad (106)$$

平方损失且 $\lambda = 0$ 时, w_j^* 是节点内平均残差。

分裂收益 (本讲义尺度)

$$\left(\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right] - \gamma \right). \quad (107)$$

LightGBM 在默认二阶叶得分尺度下报告的 `split_gain` 是该下降量的两倍; 分裂排序不变。

结论: 平方损失下, LightGBM 每轮寻找能把不同残差模式分开的树区域, 并用区域内平均残差修正已有模型。